

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 9.2c: Invoking Agents — One, Three, Ten Thousand

Zhigang Feng

2026-09-12 12:08:54

Topic 9.2c Agenda

T2b's §2 ended with three job descriptions committed to a folder. It never said what you type to make one run. This interlude answers that — and the question it leads to.

1. **Three rungs** — the only three ways to invoke an agent, and what separates them
2. **Ten thousand** — what a four-digit agent run actually consists of, and what made it bankable
3. **This deck's own ledger** — the four agents that wrote T2b's §2, the bill, and the mistakes
4. **Synthesis** — one question decides the rung

The claim this deck defends. The number of agents you run is not a capability decision. It is a *verification* decision — you may only buy scale when a wrong answer is cheap to detect.

Reading convention, as in T1b: [solid] = primary or citable; [hype] = vendor self-report. CLI flags verified against Claude Code 2.1.269, not documentation.

Three Rungs

The only three ways to invoke an agent — and who chooses in each

The Question T2b Left Open

Three job descriptions sit in a folder. Nothing yet runs them. There are exactly three ways to start one, and they differ in one thing: who chooses.



Only the third rung scales, because only it removes you from the loop. Rungs 1 and 2 cost a turn of your attention each — T2b's §3 third ceiling, the one that binds. Rung 3 trades that for a prior problem: what checks the output.

Answered next: the exhibit T2b's §4 argued for but never showed.

Rung 1 — You Describe It, the Model Chooses

T2b's §4 proved dispatch is an ordinary tool call, by argument and by experiment. It never showed you one. Here it is.

```
you > Check the derivation in src/aiyagari.py
      against docs/model.md.

# what the model emits -- one block in the same
# list as Read and Bash, nothing special about it
{ "type": "tool_use",
  "name": "Agent",
  "input": {
    "subagent_type": "math-agent",
    "description": "check the derivation",
    "prompt": "Review src/aiyagari.py against
              docs/model.md. The calibration is
              annual. Report only findings you
              can tie to a line number." } }

# ... the sub-agent's whole session happens here ...

{ "type": "tool_result",
  "content": "line 34: utility() has no domain
             guard. With SIGMA=2, u(c)=-1/c is
             positive for c<0 ..." }
```

- **You typed no agent name.** The model read the description fields in your folder and chose — the same way it chooses Read
- **prompt is the brief,** T2b's §2 step 4, written by the model this time rather than by you
- **One tool_result comes back.** Everything the sub-agent read is gone — T2b's §2 step 5

The same thing in a real transcript, from the lab: Bash → Agent code-reviewer → Read — Agent sits in the same list as the others. That is the whole claim. [solid]

Answered next: what changes when you name it yourself.

Rung 2 — You Name It

Naming the agent moves the choice from the model to you. Three surfaces do it, and this course already uses two of them without saying so.

```
# (a) in prose -- the idiom the OLG demo already uses
you > Send the pseudo-code to the domain-reviewer
      sub-agent; require it to trace one cohort's
      residual end to end before I write code.

# (b) inspect what is actually in scope
/agents

# (c) pin a whole session to one specialist
claude --agent math-agent

# (d) define one inline, with no file at all
claude --agents '{"claim-checker":
  {"description":"Find the source of every number.",
   "prompt":"You verify figures against the repo."}}'
```

/agents is the one command T2's twelve do not list; T2 owns the rest of the surface.

Answered next: the rung that removes you, and reaches four digits.

Naming it skips the only test you had. Rung 1 makes the model choose by reading your description. That choice *is* the test that the description is right — T2b's §2 step 7 is built on it.

Name the agent every time and a badly worded description never fails, so you never find out. Use rung 2 when you know which specialist you want; use rung 1 when you are still debugging the English.

Rung 3 — You Script It

Rungs 1 and 2 spend a turn of your attention per dispatch. Rung 3 spends a process instead — and it is the only rung that reaches four digits.

```
# one process per unit of work, fenced read-only
for f in source/Lec09_Agentic_AI/Lec09_T*.tex; do
  claude -p "$(cat briefs/claim_check.md) $f" \
    --output-format json \
    --allowedTools Read Grep Glob \
    --permission-mode dontAsk \
    > "out/$(basename "$f" .tex).json" &
done
wait
# or hand them to the background control plane
claude --bg "audit T2c for unsourced numbers"
claude agents          # list them
claude logs <id>       # read one
claude stop <id>       # kill one
```

Rung	Costs you	Binds at
1 implicit	a turn, each time	your attention
2 explicit	a turn, each time	your attention
3 scripted	one brief, once	tokens per minute

The brief becomes a file. At rung 3 you cannot retype it, so it must live on disk — T2b's §2 step 8, arriving by necessity rather than discipline.

Ten Thousand

What a four-digit agent run consists of — and what made it bankable

What About Ten Thousand Agents?

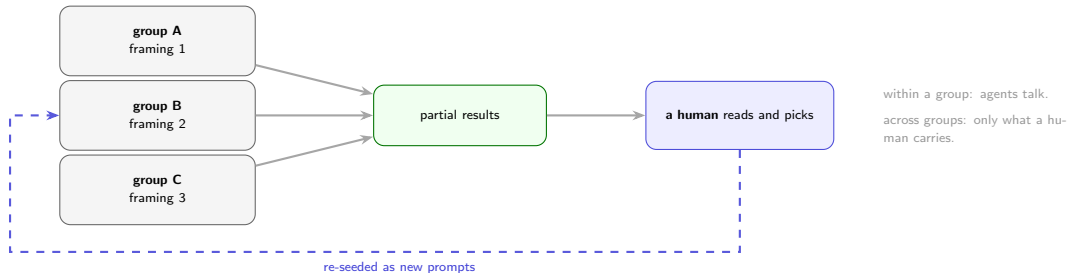
On 28 August 2026 OpenAI pointed roughly ten thousand agents at Navier–Stokes and had a Lean-checked proof 88 hours later. The number is the least interesting part.

Warm-up (Euler)	~100 agents, ~50 hours
The run (Navier–Stokes)	~10,000 agents from 28 Aug 2026, done within 88 hours
Formalisation	a further 17 hours to check the proof in Lean
Traffic	~5 million messages <i>between agents</i> ; ~300 billion output tokens
Compute	“several million dollars” (Bubeck, OpenAI)
What was shown	finite-time singularity from smooth data — the <i>negative</i> resolution

Read the ledger, not the headline. Every figure is the vendor’s own account of an unreleased model, four days old here, and the result is disputed two frames on. **[hype]** What is worth your attention is the *architecture* — reported consistently, and the part that transfers.

Not Ten Thousand Geniuses: Groups, Framings, and a Human

Not ten thousand independent attempts. Agents sat in groups that talked internally, each group got a different framing of the same problem, and a human moved fragments between groups.



What varied was dial one. Different framings of one problem *are* different system prompts — T2b's §2 step 2, run ten thousand times instead of three — wired as T2b's §3 blackboard with a person as the board. Sober reading: a thousand-agent run with better synthesis might land in the same place.

The Verifier Is the Whole Story

You may only spend money on scale when a wrong answer is cheap to detect. Lean made a wrong proof cost nothing. That, and not the ten thousand, is the part that transfers.

Reporting best-of- n as your score	the sin T1b names: τ -bench pass-hat- k falls from $\sim 90\%$ at $k=1$ to $\sim 57\%$ at $k=8$, so a best-of- n headline flatters the model and hides the reliability curve
Running n attempts behind a checker that can reject	legitimate — because the <i>checker</i> decides, not the score. The 9,999 failures cost tokens and nothing else

Two things Lean does not buy. A human must still certify that the statement checked in Lean is the theorem anyone meant to prove — the formal check is sound about the object it was given, not about your intent. And the result is contested: Buckmaster alleges OpenAI muscled in on work he and Alpöge were finishing, called one AI-written paper “AI slop” *despite* its formal verification, and OpenAI denies the charge. **[solid]** on the dispute.

Answered next: you already own a run like this — and it worked for exactly this reason.

Your Own Four-Digit Run, and Why It Worked

Lec10's T6b labelled 11,171 of 13,225 tasks in a single five-hour window. Nobody called it a swarm, but it is the same machine — and it had a verifier.

	Navier–Stokes	T6b, your labelling run
The checker	Lean: sound, mechanical	a rubric, cached per task
The hard cases	re-framed and re-seeded by a human	a Sonnet adjudication tier for the 3,456 flagged
Not repeating work	—	one cache file per task; re-runs skip
Human time	re-seeding between groups	~20 min to spec the rubric; ~12 labels read by eye

Rubric + adjudication tier + cache is a verifier — weaker than Lean, cheap enough to run 13,000 times. **And note what neither buys:** you can re-run a proof attempt or a labelling task ten thousand times, but you cannot re-run 2008. A central bank gets one history, so fan-out never manufactures a counterfactual (Lec05).

T6b owns these numbers and the bulking/caching design; this frame owns only the mapping. [solid]

This Deck's Own Ledger

The four agents that wrote T2b's §2 — the bill, and the mistakes

Who Wrote T2b's §2, and What It Cost

The thirteen frames you read in T2b's §2 were not written by one session. Four agents did the reading. Here is the bill, from this project's own logs.

Type	Brief	Tokens	Calls	Min
Explore	map the target deck and its house style	57,474	15	3.0
Explore	build system, CJK support, TikZ idioms	121,309	43	5.0
Explore	course context, capstone, PDF tooling	145,432	34	5.6
Plan	draft the frame-by-frame outline	124,361	30	10.3
total		448,576	122	23.8

448,576 tokens were read; four summaries came back — roughly twenty- to thirty-fold compression, and the orchestrating session never held the 122 file reads that produced them. That is T2b's §3 isolated-context argument with a price on it. The three explorers also ran *concurrently*: **5.6 minutes of wall clock against 13.5 sequential** (T4 owns the general speed claim).

The Design Work Was the Four Briefs

Dispatching cost one line each. What decided whether the deck was any good was what each brief asked for — T2b's §2 step 4, done four times.

Asked for	Came back	Changed
sections, frames, verbatim style patterns	no deck shows an agent file	the section's premise, and the style contract
engine, CJK support, best existing figures	a Chinese guard exists in Lec01; zero <code>includegraphics</code> ; two TikZ idioms	staying on <code>pdflatex</code> ; both card figures
capstone, prerequisites, PDF tooling	prerequisites require neither Python nor ML; a converter pipeline exists on disk	claims 1 and 4; the pipeline frame

None of the three asked a question whose answer was already known. Each asked for an *inventory* — what exists, where, quoted — which is what a read-only agent with a clean window is good at. Ask an explorer for a *judgement* and you get your own prior back with file paths attached (T5).

Where Those Four Agents Lived

Three different things get called “the agent”, and they have three different lifetimes. Only one of them was yours, and it never ran.

	Written by	Lives in	Survives?
the <i>type</i> (Explore, Plan)	the vendor, built in	the harness, delivered as a list in the system prompt	yes, but not yours to edit
the <i>brief</i>	the model , in the tool call	nowhere — four tool-call arguments	no
the <i>report</i>	the sub-agent	the message list, plus session scratch	no
code-reviewer, math-agent, ...	you	.claude/agents/ in the lab	yes, versioned

Your six agent files have never run — not once. .claude/agents/ resolves relative to the directory the session starts *in*: the only such folder here is inside the lab, and that session started at the courses root. **Where you put the folder decides whether the specialist exists** — T4's resolution rule for CLAUDE.md, applied to agents.

What Went Wrong — Four Mistakes, in Order of Interest

The ledger looks efficient. It was not. Three of these are avoidable and one is a real constraint worth knowing.

Paid twice for one answer	while the first explorer mapped the deck, the orchestrator read the same file itself. <i>Once you dispatch, stop doing the job</i>
Paid for a plan nobody waited for	the Plan agent ran 10.3 min; the outline was written before it returned. <i>Block on it, or do not spawn it</i>
Skipped step 8	four briefs written, none saved — three of them reusable across every deck here
Could not delegate what needed it	thirteen overflowing frames are thirteen independent problems, ideal fan-out — but thirteen edits target <i>one file</i>

The binding constraint was shared mutable state, not the model. The lever is the tool allowlist from step 3: read-only agents compose, writing agents serialise. Fan out workers that *return* replacement text; let one writer apply it.

Choosing a Rung Is One Question

Not “how many agents can I run” — T2b’s §3 already answered that. The question is what checks the output, and how cheaply.

What can check the output	Rung	Worked example
only you, by reading	1–2, N small	the three background specialists, judged by a 3×3 detection matrix (T2b §2)
a cheap partial check — tests, a rubric, a schema — plus a tier for the hard cases	3, N in the thousands	T6b: cached rubric, Sonnet adjudication, 11,171 labels
a sound mechanical check	3, N until the quota binds	Lean, a compiler, a one-command replication script

N is a verification choice, not a capability choice.

T2b’s §3 tells you where the ceiling is. The verifier tells you whether you want to go near it.

Bridge: Three Rungs, One Folder, and a Lab

Everything in this deck is one of two artifacts: a folder of Markdown files, or a loop over processes. Neither is a framework, and both are yours.

In the lab, added for this deck

- Dispatch the three background specialists — the prompt strings T2b's §2 asked for and never gave you
- Run one task at all three rungs and compare the transcripts
- Fan out read-only over every deck in the lecture and count unsourced numbers

Committed with this deck

- `deck-cartographer` — map a deck's structure and style
- `boundary-auditor` — say which deck owns a topic, with quotes
- `claim-checker` — find the source of every number, or flag it

The three briefs from §3 that should have been files. At the repository root, so they are in scope where you actually work.

Then back to the spine. T3 returns to method — the homotopy workflow that decides *what* you should be asking any of these agents to do. A rung is a delivery mechanism; it does not improve the question.